

Вопросы

по дисциплине «Компьютерное моделирование»,
2026

1. Введение в корпоративную разработку. Отличия от учебных проектов. Роли в команде. Жизненный цикл ПО.

Это процесс создания программного обеспечения в условиях жестких ограничений: бюджет, сроки, масштабируемость и безопасность. Это не про "написать код", а про решение бизнес-задачи ценой оптимальных ресурсов.

Ключевые принципы:

- Надежность (SLA): Система должна работать 24/7 (например, 99.99% времени).
- Безопасность: Защита данных клиентов (GDPR, ФЗ-152).
- Масштабируемость: Способность выдержать рост нагрузки (например, в Черную пятницу).
- Поддерживаемость: Код пишут один раз, а читают и правят годы.

Отличия от учебных проектов (Критично важно)

Критерий	Учебный проект	Корпоративный проект
Цель	Получить оценку, сдать экзамен.	Заработать деньги, оптимизировать затраты, удержать клиента.
Сроки	Жесткий дедлайн, после сдачи проект "умирает".	Бесконечный цикл. Релизы выходят раз в 2 недели / месяц.
Команда	Работаете один или в равной группе.	Четкая иерархия и разделение зон ответственности.
Качество кода	"Лишь бы работало".	Чистота кода критична (Code Review, SonarQube). Технический долг контролируется.
Инфраструктура	Локальный компьютер (localhost).	CI/CD (автосборка), Docker, облака (AWS/Yandex), мониторинг (Prometheus).
Данные	Тестовые данные (5-10 записей).	Миллионы/миллиарды строк в БД. Запросы должны быть оптимизированы.
Ошибки	"Упало — перезапустил".	"Упало" = потеря денег. Нужны логи, трейсы и быстрый откат (Rollback).
Документация	Курсовая работа ради объема.	Живая документация (OpenAPI, Swagger, Confluence) для онбординга новых сотрудников.

Роли в команде (Enterprise Roles)

В корпоративной разработке редко бывает просто "программист". Есть RACI-матрица (ответственные, исполнители, консультанты, информируемые). Основные роли:

1. Владелец продукта (Product Owner / Заказчик)

- Представляет бизнес. Определяет, ЧТО нужно делать.

2. Project Manager / Scrum Master

- Отвечает за КАК мы работаем и соблюдение сроков.
- Решает организационные проблемы, проводит митинги, защищает команду от внешних помех.

3. Team Lead / Архитектор

- Технический лидер. Принимает стратегические решения: какой фреймворк, как делить систему на микросервисы, как хранить данные.
- Проводит сложные Code Review.

4. Разработчики (Devs)

- Backend: Пишут бизнес-логику, API, работу с БД.
- Frontend: Пишут интерфейсы (Web, Mobile).
- Fullstack: Микс первых двух (часто в стартапах).
- QA (Инженер по тестированию): Пишет автотесты, ищет баги. Бывает Manual (ручной) и Automation (пишет код тестов).

5. DevOps / SRE (Site Reliability Engineer)

- Отвечает за инфраструктуру: сервера, сети, базы данных, развертывание (Deploy), мониторинг.
- Девиз: "Вы пишете код, я гарантирую, что он полетит в прод".

6. Аналитик (Business/System Analyst)

- Переводчик между бизнесом и разработчиками. Пишет детальные Технические Задания (ТЗ), описывает сценарии использования (Use Cases).

Жизненный цикл ПО (SDLC - Software Development Life Cycle)

Это стандартный процесс, который проходит программа от идеи до удаления. Классическая структура (независимо от методологии):

Этап 1: Анализ требований (Requirements)

Сбор пожеланий заказчика. Формирование MVP (Minimum Viable Product) — минимально жизнеспособного продукта.

Результат: Документ с функциональными требованиями.

Этап 2: Проектирование (Design)

Создание архитектуры: UML-диаграммы, выбор БД (SQL/NoSQL), протоколов связи (REST/gRPC), схемы БД.

Результат: Дизайн-документ (High-Level Design).

Этап 3: Разработка (Implementation / Coding)

- Непосредственное написание кода.
- Используется система контроля версий (Git) и принят Git Flow (ветки: main, develop, feature/..., hotfix/...).

Этап 4: Тестирование (Testing)

- Unit-тесты (проверка мелких функций).
- Интеграционные тесты (проверка связи модулей).
- E2E (End-to-End) тесты (проверка всего пользовательского сценария).
- Результат: Отчет о дефектах (Bug Report).

Этап 5: Развертывание (Deployment / Release)

- Выкатка в эксплуатационную среду (Production).
- Современный подход — CI/CD (Continuous Integration / Continuous Delivery). Код автоматически собирается, тестируется и выкатывается при пуше в репозиторий.

Этап 6: Эксплуатация и поддержка (Maintenance)

Самый долгий этап.

Мониторинг (смотрим логи, CPU, память).

Исправление критических багов (Hotfix).

Добавление новых фич (это возврат к этапу 1, цикл замыкается).

Этап 7: Вывод из эксплуатации (Retirement)

Когда система устарела, данные мигрируют в новую, а старая отключается.

2. Монолит vs. Микросервисы vs. Гибрид (микросервисный монолит). Плюсы, минусы, критерии выбора.

Это когда весь код (фронтенд, бэкенд, работа с БД, бизнес-логика) собирается в один единый исполняемый файл (.jar, .war, .exe) и деплоится на один сервер.

Плюсы (+)

- Простота разработки: Легко запустить локально, легко писать интеграционные тесты
- Быстрый деплой на старте: Один билд — один релиз. Легкий рефакторинг: IDE легко находит все использования методов, потому что код един.

Минусы (-)

Ада удаленного деплоя: Чтобы поправить одну строчку в калькуляторе скидок, нужно пересобрать и передеплоить весь проект (десятки гигабайт).

Зоопарк зависимостей: Одна библиотека (Hibernate) версии 4.0 и 5.0 не могут жить в одном монолите. Конфликты зависимостей — головная боль.

Масштабирование целиком: Если у вас в 5 раз выросла нагрузка на модуль "Оплата", вы не можете масштабировать только его. Вы масштабируете весь монолит (включая всю админку и логи), что дорого.

Технологическое рабство: Вы пишете на Java 8 — вся команда пишет на Java 8. Воткнуть туда Python-модуль для машинного обучения невозможно.

Микросервисы (Microservices)

Это когда вы режете приложение на множество маленьких независимых сервисов (по 1–2 разработчика на сервис). Каждый сервис имеет свою базу данных и общается с другими по сети (REST, gRPC, Kafka).

Плюсы (+)

Независимый деплой: Исправили "Корзину" — деплоим только "Корзину". Остальное не трогаем.

Технологическая свобода: Сервис "Рекомендации" можно написать на Python (для AI), а "Платежи" на Go (для высокой производительности) — и они будут дружить.

Гранулярное масштабирование: Нагружены только "Поиск" и "Каталог"? Масштабируем только их, не тратя деньги на "Личный кабинет".

Минусы (-)

Распределенный ад (Fallacies of distributed computing): Сеть ненадежна. Сервис А вызвал Сервис Б, Б вызвал В, В упал. Что делать? Транзакции распределенные — это очень сложно (Saga, 2PC).

Операционная сложность: Вам нужен топ-DevOps. Вместо 1 базы данных у вас 10. Вместо 1 лога — 10 логов, которые нужно собирать в ELK.

Сложность тестирования: Запустить 10 сервисов у себя на ноутбуке для отладки — памяти не хватит.

Гибрид (Микросервисный монолит / Modular Monolith)
Важнейший тренд последних лет. Это архитектура, при которой вы пишете как монолит (один репозиторий, один деплой), но внутри организуете код как микросервисы (разбиваете на слабосвязанные модули с четкими границами).

Каждый модуль имеет свой API, свою схему БД, и общается с другими модулями через события, а не прямой вызов методов (используя внутреннюю шину сообщений, но без сети).

Плюсы (+)

Скорость старта: Нет проблем с сетевыми вызовами и деплоем 100 подов в Kubernetes.

Готовность к миграции: Если модуль "Оплата" станет слишком тяжелым, вы просто "вырезаете" его код, заворачиваете в отдельный микросервис, а остальные модули переключаете на REST/gRPC вместо внутреннего вызова. Миграция занимает часы, а не годы.

Строгие границы: Препятствует "спагетти-коду", так как модули не могут напрямую лазить в таблицы друг друга.

Минусы (-)

Сложность внедрения: Требуется железной дисциплины от команды (нельзя просто так взять и вызвать метод другого модуля — нужно через события).

Один язык программирования: Вы все еще привязаны к одной экосистеме (например, Java/Kotlin).

3. Архитектурные стили и паттерны: Клиент-сервер, Layered Architecture (n-слойная), Event-Driven Architecture (EDA). Обзор паттернов (например, MVC, CQRS на высоком уровне)

Клиент-Сервер (Client-Server)

Это базовый, самый старый и вездесущий стиль. Описывает простое правило: есть потребитель (Клиент) и есть поставщик ресурсов (Сервер).

Суть: Клиент инициирует запрос, Сервер обрабатывает и отдает ответ. Клиенты никогда не общаются напрямую друг с другом.

Пример: Браузер (клиент) -> Веб-сервер (nginx) -> Сервер приложений (Backend).

Главный нюанс: Сервер — это «толстый» мозг, клиент — «тонкий» экран. Вся бизнес-логика лежит на сервере.

Layered Architecture (n-слойная / Многоуровневая)

Это «золотой стандарт» для 90% корпоративных приложений. Код делится на горизонтальные слои, где каждый слой знает только о том, что на-ся строго под ним.

Классическая схема (3-слойка / 4-слойка):

Presentation Layer (UI/API): Контроллеры, REST-методы. Отвечает за ввод/вывод данных. Не содержит логики, только парсинг JSON и валидацию входящих данных.

Business Logic Layer (BLL / Service Layer): Святое место. Здесь живет вся суть бизнеса (расчет скидок, проверка баланса, бизнес-правила).

Persistence Layer (DAO / Repository): Отвечает за общение с базой данных (SQL-запросы, ORM — Hibernate/Entity Framework).

Database (БД): Физическое хранилище.

Event-Driven Architecture (EDA) — Событийно-ориентированная

Это альтернатива синхронному запросу. Вместо того чтобы Сервис А звал Сервис Б по REST и ждал ответа (блокируя поток), Сервис А просто кричит в пустоту: "Случилось событие!" — и забывает об этом.

Ключевые элементы:

Producer (Издатель): Тот, кто создает событие (например, OrderCreated).

Message Broker (Брокер): Посредник (Kafka, RabbitMQ, AWS SQS). Он хранит событие.

Consumer (Подписчик): Тот, кто слушает брокера и реагирует на событие (например, сервис "Почта" подписался на OrderCreated, чтобы отправить письмо).

Плюсы (+):

Асинхронность: Сервисы не тормозят друг друга. Пользователь нажал "Купить" и сразу получил ответ "Заказ принят", даже если письмо идет 5 секунд.

Минусы (-):

Сложность отладки: Трассировка запроса превращается в квест (нужно смотреть ID события через логи всех сервисов).

Обзор ключевых паттернов (MVC и CQRS)

Эти паттерны часто путают или считают взаимозаменяемыми, но они решают разные задачи.

A. MVC (Model-View-Controller) — Архитектура пользовательского интерфейса

Это стандарт для веб-фреймворков (Spring MVC, ASP.NET Core, Django).

Model: Ваши бизнес-данные и правила (POJO / Entity).

View: То, что видит пользователь (HTML-шаблоны или JSON-ответ).

Controller: Диспетчер. Принимает HTTP-запрос, дергает нужный метод Сервиса (Model), получает данные и говорит View, как их отобразить.

CQRS (Command Query Responsibility Segregation) — Разделение на Чтение и Запись

Это продвинутый паттерн для высоконагруженных систем (Amazon, Aviasales).

В обычной жизни у нас одна модель данных (Entity) используется и для записи, и для чтения. CQRS говорит: «Это две разные модели!»

- Command (Команда): Изменяет состояние системы (INSERT, UPDATE, DELETE). Работает с Записывающей БД.
- Query (Запрос): Только читает данные (SELECT). Работает с Читающей БД (часто это кэш или подготовленные денормализованные таблицы).

4. Методологии управления проектами. Каскадная модель.

Что такое Каскадная модель (Waterfall)?

Это линейная последовательная модель. Вы не можете перейти на следующий этап, пока полностью не завершен предыдущий. Название произошло из-за того, что поток работ идет строго вниз, как вода с уступа на уступ (requirements -> design -> implementation -> testing -> deployment).

Классические этапы Waterfall (по Г. Бучу и Р. Уинстону):

- Анализ и сбор требований (Requirements): Встречи с заказчиком, написание ТЗ (Технического Задания) на 100+ страниц. Результат — SRS (Software Requirements Specification), который подписывается обеими сторонами.
- Проектирование (System Design): Архитекторы пишут документ HLD (High-Level Design) и LLD (Low-Level Design): схемы БД, протоколы, классы, интерфейсы.
- Разработка (Implementation): Программисты пишут код строго по утвержденному дизайну. (Если дизайн не учитывает какую-то фичу — пиши письмо, вноси правку в ТЗ).
- Тестирование (Testing): Выкатка на QA-стенд. Тестировщики ищут баги. Важно: Если баг найден на этом этапе, исправление стоит очень дорого, потому что нужно переписывать код, иногда пересматривать дизайн.
- Внедрение (Deployment): Передача заказчику, установка на его оборудование.
- Эксплуатация и поддержка (Maintenance): Исправление косяков, которые обнаружились в боевой среде.

Плюсы Waterfall (Главные аргументы "за")

Дисциплина и порядок. Идеально для государственных контрактов и банковских систем, где важна документация. Вы точно знаете, кто, когда и за сколько делает.

Четкая стоимость (Fixed Price). Так как требования зафиксированы, можно точно посчитать бюджет.

Простота управления. Ты закончил этап А -> перешел к этапу Б. Не нужно синхронизировать 10 команд в спринте.

Критические Минусы

"Бонинг-эффект" (Антипаттерн анализа): Заказчик не знает, чего хочет, пока не увидит работающий продукт. А Waterfall требует подписать ТЗ в самом начале

Гигантский провал в сроках: Ошибка в требованиях, найденная на этапе тестирования (через 8 месяцев), стоит в 100 раз дороже, чем исправление её в начале.

Риск устаревания: Пока вы пилите монолит 2 года, конкуренты уже выпустили 10 релизов. К моменту выхода ваш продукт может быть уже неактуален рынку.

5. Методологии управления проектами. Scrum (роли, артефакты, события)

Scrum — это не инструмент (не Jira) и не набор правил (не «делай так и будет счастье»). Это фреймворк для решения сложных проблем, когда заранее неизвестно идеальное решение.

Три "Кита" (Эмпиризм Scrum)

В основе Scrum лежит эмпирический подход — знание через опыт. Это работает за счет трех принципов:

Прозрачность (Transparency): Каждый видит, что происходит. Кто что делает, какие задачи заблокированы, какой бюджет.

Инспекция (Inspection): Команда часто останавливается и смотрит на свой продукт и процесс.

Адаптация (Adaptation): Если инспекция показала, что мы идем не туда или делаем медленно — мы меняем свои действия немедленно, а не ждем окончания проекта.

2. Роли в Scrum (Их всего ТРИ)

В классическом Scrum нет "Тимлида" и "Архитектора" как отдельных ролей (это навыки внутри Dev Team). Есть строго 3 роли:

- **A. Product Owner (PO) — «Голос бизнеса»**
 - Ответственность: Максимизировать ценность продукта.
 - Чем занимается: Управляет Бэклогом (Backlog) — расставляет приоритеты. Решает, ЧТО будем делать в следующем спринте.
- **Б. Scrum Master (SM) — «Слуга-лидер» и «Полицейский процесса»**
 - Ответственность: За соблюдение правил Scrum и устранение помех.
 - Чем занимается: Проводит ретроспективы, учит команду самоорганизации, защищает команду от внешнего шума

Артефакты Scrum (Что мы ведем)

A. Product Backlog (Бэклог продукта)

- Упорядоченный список всего, что когда-либо может понадобиться в продукте.
- Это живой документ. Он постоянно пополняется новыми идеями и фичами.

Б. Sprint Backlog (Бэклог спринта)

- Это подмножество Product Backlog, которое команда выбрала делать в текущем спринте.
- Это не просто список, это план-схема. Задачи разбиты на часы/дни, и команда видит, кто что делает.

В. Инкремент (Increment)

- Сумма всех завершенных элементов бэклога за текущий спринт плюс все, что было сделано в предыдущих спринтах.
- Ключевое правило: Инкремент в конце каждого спринта должен быть работающим (Done) и готовым к выкатке в прод. Не "почти готово", а "можно релизить".

4. События (Церемонии) Scrum — Временные боксы

В Scrum все события имеют жесткий лимит времени (Timebox). Это чтобы не засиживаться на совещаниях.

1. Sprint (Спринт) — Сердце Scrum

- Это контейнер для всех остальных событий. Временной отрезок, в течение которого создается инкремент.
- Длительность: Обычно 1–4 недели (классика — 2 недели).
- Иммуниет: Во время спринта запрещено менять его цель и состав задач (если только не форс-мажор).

2. Sprint Planning (Планирование спринта) — Вход в спринт

- Время: До 8 часов для месячного спринта.
- Что делаем: Команда и РО берут самые приоритетные задачи из Product Backlog и решают, какой объем они реально успеют сделать (основываясь на своей Velocity — скорости).
- Результат: Sprint Backlog и цель спринта (Sprint Goal) — краткое описание того, зачем мы это делаем.

3. Daily Scrum (Ежедневная синхронизация / Stand-up)

- Время: Ровно 15 минут. Каждый день, в одно и то же время. Стоя (чтобы не задерживаться).
- Главные вопросы (античные, но работают):
 1. Что я сделал вчера для достижения цели спринта?
 2. Что я сделаю сегодня?

6. Методологии управления проектами. Scrum (Спринты, бэклог, планирование) Product Backlog (Бэклог продукта)

Это полный список всего, что когда-либо может понадобиться в продукте. Это не просто «список задач» — это стратегический план развития продукта на месяцы/годы вперед. Как устроен?

- Задачи расположены строго по приоритету (самое важное — сверху).
- Верхние задачи детально проработаны (готовы к разработке), нижние — только в виде идей.
- Это живой документ — он постоянно обновляется: что-то добавляется, что-то удаляется, что-то меняет приоритет.

Кто управляет?

Только Product Owner (PO). Он — голос бизнеса, он решает, что важнее для рынка и клиентов.

Sprint Planning (Планирование спринта)

Это встреча, где команда и РО договариваются, какие задачи из бэклога мы делаем в ближайшие 2 недели. Длится 2–4 часа.

Как проходит? (3 шага):

Шаг	Что происходит	Пояснение
1.Считаем мощность	Смотрим на Velocity (сколько стори поинтов команда делала в среднем за прошлые спринты) и Capacity (сколько человеко-часов реально есть с учетом отпусков и больничных).	Это чтобы не брать слишком много или слишком мало. Прогнозируемость важна для бизнеса.
2.Выбираем задачи	РО показывает самые приоритетные задачи из бэклога и объясняет, почему они важны для бизнеса.	Команда понимает ценность работы, а не просто «делаем потому что сказали».
3.Команда оценивает берет обязательств	Разработчики сами решают, сколько задач они реально успеют сделать качественно. Никто не назначает задачи сверху!	РО говорит «ЧТО делать», а команда говорит «СКОЛЬКО мы успеем». Это договор, а не приказ.

Sprint (Спринт)

Это временной отрезок (обычно 2 недели), внутри которого команда создает готовый инкремент продукта.

Главные правила:

Правило	Пояснение
Фиксированная длина	Спринт всегда одной длины (например, ровно 2 недели).
Состав задач не меняется	Как только спринт начался — нельзя добавить новую задачу или поменять приоритеты.
Конечный результат	В конце спринта должен быть работающий инкремент — готовый к выкатке продукт (даже если бизнес решает не выкатывать его сразу).

Backlog Refinement (Груминг / Уточнение бэклога)

Это отдельная встреча (обычно 1–2 часа раз в неделю), где команда с РО смотрят на задачи из бэклога, которые запланированы на будущие спринты.

Что делают:

- Крупные задачи (эпики) разбивают на маленькие, чтобы они помещались в спринт.
- Уточняют требования и технические детали.
- Предварительно оценивают трудоемкость (чтобы на планировании не гадать).

Зачем это нужно?

Чтобы на Sprint Planning вы не тратили 4 часа на выяснение «а что же тут имелось в виду». К моменту планирования топ-10 задач уже должны быть полностью прозрачными и понятными.

7. Методологии управления проектами. Kanban

Kanban (канбан) — это визуальный метод управления работой, который фокусируется на непрерывном потоке задач и ограничении незавершенной работы (WIP — Work In Progress). **Откуда взялся:**

Из производственной системы Toyota (точно вовремя / Just-in-Time). В IT пришел как способ визуализировать работу на доске и не перегружать команду. **Главная идея:** Не "делать больше", а делать плавно. Мы не делим работу на спринты, а постоянно берем новые задачи, как только освобождается место.

Базовые принципы Kanban (4 фундаментальных правила)

Принцип	Что значит	Пояснение
1.Визуализируйте поток работы	Сделайте задачи видимыми.	Используем доску (физическую или цифровую) с колонками: To Do → In Progress → Review → Done. Каждая задача — это карточка.
2.Ограничьте количество незавершенной работы (WIP)	Установите жесткие лимиты на колонки.	Например, в колонке "In Progress" может быть не более 3 задач. Нельзя взять новую, пока не закончишь одну из текущих.
3.Управляйте потоком	Следите за тем, как задачи движутся по доске.	Если задачи застревают в колонке "Review" — значит, проблема в процессе. Нужно это исправлять.
4.Постоянно улучшайтесь	Анализируйте метрики и оптимизируйте.	Используем Cycle Time и Lead Time, чтобы находить узкие места и устранять их.

Основные элементы Kanban

Элемент	Объяснение
Kanban-доска	Разделена на колонки, соответствующие этапам работы (например: Backlog → Analysis → Development → Testing → Done).
Карточки (задачи)	Каждая задача — это карточка, которая перемещается по доске слева направо по мере выполнения.
CFD (Cumulative Flow Diagram)	График, показывающий, сколько задач находится на каждом этапе. Помогает увидеть "пробки" в процессе.
Lead Time	Время от появления задачи в бэклоге до её завершения.
Cycle Time	Время от начала работы над задачей (колонка "In Progress") до её завершения.

Ключевые отличия Kanban vs Scrum

Это самый частый вопрос на экзамене/собеседовании!

Критерий	Scrum	Kanban
Ритм работы	Работа делится на спринты(фиксированной длины, 1–4 недели).	Работа идет непрерывно, нет спринтов.
Роли	Есть строгие роли: Product Owner, Scrum Master, команда.	Нет строгих ролей — только команда, менеджер может быть опционально.
Планирование	Планирование в начале каждого спринта. Команда берет обязательства на спринт.	Планирование по мере необходимости. Когда появляется свободная мощность — берем следующую задачу.
Когда использовать	Когда нужна предсказуемость и стабильный ритм, требования меняются часто, но внутри спринта — заморозка.	Когда задачи поступают неравномерно (например, поддержка, инциденты), когда нужна гибкость и быстрая реакция.

8. Методологии управления проектами. Scrumban

Scrumban — это гибридная методология, которая берет лучшее из Scrum (планирование, роли, ретроспективы) и лучшее из Kanban (непрерывный поток, WIP-лимиты, гибкость). Откуда взялся:

Когда команды поняли, что чистый Scrum слишком жесткий (спринты, заморозка задач), а чистый Kanban слишком хаотичный (нет структуры), они начали смешивать подходы. Scrumban — это эволюционный переход от Scrum к Kanban (или наоборот).

Главная идея: Работаем непрерывным потоком (как в Kanban), но с элементами планирования и улучшения (как в Scrum). Нет жестких спринтов, но есть регулярные встречи для синхронизации.

2. Базовые принципы Scrumban (что взяли откуда)

Откуда	Что взяли	Пояснение
Из Scrum	Планирование (Sprint Planning)	Проводим регулярные плановые встречи, чтобы разбирать бэклог и определять приоритеты.
Из Scrum	Ретроспектива (Retrospective)	Регулярно анализируем процесс и улучшаем его (раз в 1–2 недели).
Из Scrum	Роли (Product Owner, команда)	Есть РО, который управляет бэклогом. Scrum Master может быть опционально.
Из Kanban	WIP-лимиты	Ограничиваем количество задач в работе, чтобы не перегружать команду.
Из Kanban	Непрерывный поток	Нет спринтов — задачи берутся по мере освобождения мощности.
Из Kanban	Гибкость	Можно менять приоритеты в любой момент, добавлять срочные задачи.

Ключевые элементы Scrumban

Элемент	Объяснение
Бэклог	Упорядоченный список задач (как в Scrum). Управляется Product Owner.
Канбан-доска	Колонки, отражающие этапы работы (например: Ready → Dev → Review → Done).
WIP-лимиты	Ограничения на количество задач в колонках (как в Kanban).
Планирование	Регулярные встречи (раз в 1–2 недели) для уточнения задач и приоритетов.
Ретроспектива	Регулярные встречи для улучшения процесса.
Метрики	Используются Lead Time / Cycle Time (как в Kanban), а не Velocity (как в Scrum).

Преимущества Scrumban

Плюс	Минус
Гибкость	Нет предсказуемости релизов
Нет стресса спринта	Нет четкой структуры
Предсказуемость	
Эволюционный переход	Сложно внедрить
Меньше переключений	Новички могут
Прозрачность	РО может злоупотреблять гибкостью

9. Системы трекинга задач и ошибок. Цели и принципы

Это информационная система для регистрации, хранения, назначения и контроля исполнения задач, багов (ошибок) и запросов на изменения в процессе разработки ПО.

Простыми словами: Это единое место, где «живет» вся работа команды. Если нет записи в трекере — работы не было.

Главные ЦЕЛИ внедрения таких систем

Цель	Объяснение
Централизация информации	Вся коммуникация по задаче (обсуждения, файлы, ссылки, код) хранится в одном месте. Никаких «а где у нас этот макет? Он в почте или в телеграме?».
Прозрачность (Visibility)	Любой член команды (и даже заказчик) в любой момент видит: кто, что, когда и в каком статусе делает. Это убивает слухи и лишние вопросы.
Приоритизация	Помогает бизнесу (Product Owner) расставлять задачи по важности, чтобы команда делала сначала самое ценное, а не то, что громче кричит.

Популярные системы трекинга (Краткий обзор)

Система	Для чего хороша	Особенность
Jira (Atlassian)	Стандарт индустрии. Для больших команд и сложных проектов.	Гибкая настройка workflow, мощные отчеты, интеграция с Confluence и Bitbucket.
YouTrack (JetBrains)	Для Agile-команд.	Удобный поиск, настраиваемая доска, хорош для разработчиков, любящих хоткеи.
Trello	Для небольших проектов и стартапов.	Простая доска с карточками (Kanban), минимализм, легко начать.
Azure DevOps (Microsoft)	Для экосистемы .NET и глубокой интеграции с кодом.	Совмещает репозитории (Git), CI/CD и трекинг в одном окне.

10. Системы контроля версий (VCS).
Централизованные vs распределенные.

Это инструмент, который отслеживает изменения в файлах (обычно в исходном коде) на протяжении времени. Он позволяет:

- Вернуться к любой старой версии файла.
- Смотреть историю: кто, когда и зачем изменил строку кода (аннотации / Blame).
- Работать в команде, не затирая код друг друга (слияния / Merge).

Простыми словами: Это «бесконечная кнопка Ctrl+Z» для всего проекта, плюс это «Фейсбук» для разработчиков (видно, кто что делал).

Главная классификация: Централизованные (CVCS) vs Распределенные (DVCS)

Разница в том, где хранится полная история проекта.

Критерий	Централизованные (CVCS)	Распределенные (DVCS)
Где хранится история?	Только на одном сервере.	На сервере И на каждом компьютере разработчика (полная копия).
Примеры	SVN (Subversion), CVS, Perforce.	Git, Mercurial (Hg).
Нужен ли интернет?	Да. Без связи с сервером нельзя посмотреть историю, сделать коммит или сравнить версии.	Нет. 99% операций (коммит, просмотр логов, сравнение) делаются локально, без интернета.
Скорость	Медленная (каждая команда идет на сервер по сети).	Очень быстрая (все операции в оперативной памяти/диске локально).
Риск потери данных	Высокий. Если сервер упал или стерлась БД — проект потерян (если нет бэкапа).	Низкий. У каждого разработчика есть полный бэкап всего репозитория. Сервер можно восстановить с любого компьютера.
Работа с ветками	Ветки (Branches) и медленные. Создавать их страшно, часто работают в одной ветке (Trunk).	Ветки — легковесные (весьт пару байт). Git поощряет создавать ветку под каждую задачу (Feature Branch).
Контроль доступа	Жесткий централизованный (админ решает, кто что видит).	Гибкий: можно делать Pull Request'ы, код-ревью перед вливанием.

Сравнительная таблица (Итог)

Характеристика	SVN (Централизованная)	Git (Распределенная)
Хранение истории	На сервере	На сервере + локально у каждого
Коммит	Требуется соединения с сервером	Делается локально (мгновенно)
Создание веток	Дорого, медленно, копирует папки	Дешево, быстро (указатель на коммит)
Слияние (Merge)	Сложное, часто с ошибками	Умное, трехстороннее слияние
Популярность	Устаревает (используется в гос/банковских Legacy-проектах)	Стандарт индустрии (GitHub, GitLab, Bitbucket)

11. Введение в CI/CD. Понятия Continuous Integration, Continuous Delivery. Цели, преимущества

CI/CD — это набор практик и пайплайнов (конвейеров) автоматизации, которые обеспечивают быструю и надежную поставку кода от разработчика до конечного пользователя.

Простыми словами: Это «конвейер на фабрике кода». Как только разработчик заливает код в репозиторий (Git), автоматически запускается цепочка действий: сборка → тесты → проверка качества → выкладка на сервер.

Разбираем по буквам: CI (Continuous Integration) Непрерывная ИНТЕГРАЦИЯ. Это практика, при которой разработчики несколько раз в день сливают (merge) свой код в общую ветку (обычно main или develop).

Что происходит после слияния (автоматически):

1. Сборка (Build): Система компилирует проект, чтобы убедиться, что он вообще собирается.
2. Прогон автотестов (Unit / Integration): Запускаются тесты, чтобы проверить, что новый код не сломал старый функционал (регрессии).
3. Статический анализ (Linter / SonarQube): Проверка качества кода (насколько он грязный, есть ли потенциальные баги).

Главная ЦЕЛЬ CI: Обнаружить проблему (баг или конфликт) как можно раньше, в тот же день, когда разработчик написал код, а не через месяц на стейджинге.

Разбираем по буквам: CD (Continuous Delivery / Deployment)

Здесь есть два близких понятия. Важно их разделить!

Термин	Что значит	Пример
Continuous DELIVERY (Непрерывная доставка)	Код автоматически собирается, тестируется и готов к релизу, но финальный выкат в прод (Production) происходит вручную (по кнопке).	Команда завершила спринт, нажала кнопку "Deploy" в Jenkins, и новая версия полетела на сервер.
Continuous DEPLOYMENT (Непрерывное развертывание)	Код автоматически проходит весь пайплайн вплоть до выката в прод. Без ручного подтверждения.	Разработчик залил фикс в main, через 15 минут этот фикс уже работает у миллионов пользователей.

Популярные инструменты CI/CD (Упомянуть вскользь)

- Jenkins — классика, бесплатный, настраиваемый «швейцарский нож».
- GitLab CI / GitHub Actions — встроенные CI/CD прямо в репозиторий (сейчас самый популярный тренд).
- TeamCity / Bamboo — коммерческие решения для крупных корпораций.